**ASYNTAX CODING SCHOOL**
Backend Development Course

# ASSESSMENT 02

## Employee Portal — Full Backend System

| Assessment | Total Marks | Stack | Submission |
|---|---|---|---|
| **02** | **100** | **Node.js / Express** | **GitHub + Report** |

*Read all instructions carefully before you begin.*

# 1. Overview & Learning Objectives

This assessment tests your ability to design and build a production-grade RESTful backend using Node.js with ES6 modules, following a clean architecture pattern. You will build an Employee Portal with four CRUD modules, a secure JWT authentication system, layered middleware, Supabase as the database ORM, and Upstash Redis for caching.

📌 All code MUST use ES6 import/export syntax (no require / module.exports).
📌 Use "type": "module" in package.json to enable ES6 modules in Node.js.
📌 Upstash Redis is used for caching — it is free-tier friendly and HTTP-based (no local Redis server needed).

## Learning Objectives

- Implement a layered clean architecture: Routes → Controllers → Services → Repositories → Models.
- Build secure authentication with JWT (HS256) and bcrypt password hashing.
- Use Supabase JS client as the ORM for all database interactions.
- Cache read-heavy endpoints using Upstash Redis with the cache-aside pattern.
- Write a reusable AppError class and a global error handling middleware.
- Build a standardised API response handler used consistently across all modules.
- Apply CORS, authentication guard, and input sanitiser middleware correctly.

# 2. Mandatory Technology Stack

| Category | Package | Purpose |
|---|---|---|
| **Runtime** | `Node.js v18+` | ES6 module support, server runtime |
| **Framework** | `express` | HTTP routing and middleware |
| **Database / ORM** | `@supabase/supabase-js` | PostgreSQL via Supabase client |
| **Auth** | `jsonwebtoken, bcryptjs` | JWT signing, password hashing |
| **Caching** | `@upstash/redis` | Serverless Redis (free tier) |
| **CORS** | `cors` | Cross-origin request policy |
| **Sanitiser** | `express-mongo-sanitize, xss-clean` | Input cleaning |
| **Validation** | `joi` | Schema-based request validation |
| **Config** | `dotenv` | Environment variable loading |
| **Dev** | `nodemon` | Hot reload during development |

# 3. Required Project Structure (Clean Architecture)

Your project must follow the layered folder structure below. Each layer has a single responsibility. Business logic lives only in services; data access lives only in repositories.

```
employee-portal/
├── src/
│   ├── config/
│   │   ├── supabase.js          # Supabase client singleton
│   │   └── redis.js             # Upstash Redis client singleton
│   │
│   ├── middlewares/
│   │   ├── auth.middleware.js   # JWT verification + blacklist check
│   │   ├── cors.middleware.js   # CORS policy configuration
│   │   ├── sanitise.middleware.js  # Input sanitiser
│   │   └── validate.middleware.js  # Joi validation factory
│   │
│   ├── models/                  # Field definitions / schema docs (no DB logic)
│   │   ├── employee.model.js
│   │   ├── department.model.js
│   │   ├── role.model.js
│   │   └── attendance.model.js
│   │
│   ├── repositories/            # All Supabase queries live here
│   │   ├── employee.repository.js
│   │   ├── department.repository.js
│   │   ├── role.repository.js
│   │   ├── attendance.repository.js
│   │   └── auth.repository.js
│   │
│   ├── services/                # Business logic only — no req/res
│   │   ├── employee.service.js
│   │   ├── department.service.js
│   │   ├── role.service.js
│   │   ├── attendance.service.js
│   │   └── auth.service.js
│   │
│   ├── controllers/             # Thin — parse req, call service, send response
│   │   ├── employee.controller.js
│   │   ├── department.controller.js
│   │   ├── role.controller.js
│   │   ├── attendance.controller.js
│   │   └── auth.controller.js
│   │
│   ├── routes/                  # Route definitions only
│   │   ├── employee.routes.js
│   │   ├── department.routes.js
│   │   ├── role.routes.js
│   │   ├── attendance.routes.js
│   │   ├── auth.routes.js
│   │   └── index.js             # Root router — mounts all sub-routers
│   │
│   └── utils/
│       ├── AppError.js          # Custom error class
│       ├── catchAsync.js        # Async error wrapper
│       ├── responseHandler.js   # sendSuccess / sendError helpers
│       └── errorHandler.js      # Global Express error middleware
│
├── app.js                       # Express app setup (middleware, routes)
├── server.js                    # Entry point — starts HTTP server
├── .env
├── .env.example
├── .gitignore
├── package.json                 # "type": "module" required
└── README.md
```

## 4. ES6 Module Setup

You must use ES6 import/export throughout. CommonJS (require) is not permitted. Add the following to your package.json:

```
// package.json (required fields)
{
  "name": "employee-portal",
  "version": "1.0.0",
  "type": "module",
  "main": "server.js",
  "scripts": {
    "start":   "node server.js",
    "dev":     "nodemon server.js"
  }
}
```

```
// server.js — Entry point
import 'dotenv/config';
import app from './app.js';

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

```
// app.js — Express setup
import express          from 'express';
import { corsOptions }  from './src/middlewares/cors.middleware.js';
import { sanitiseInputs } from './src/middlewares/sanitise.middleware.js';
import router           from './src/routes/index.js';
import { globalErrorHandler } from './src/utils/errorHandler.js';

const app = express();

app.use(corsOptions);
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(...sanitiseInputs);

app.use('/api', router);

// Global error handler — MUST be last middleware
app.use(globalErrorHandler);

export default app;
```

# 5. Utilities — Error Handling & Response Handler

These four utility files are the foundation of the entire backend. Write them first before any feature module.

## 5.1 AppError.js — Custom Error Class

```
// src/utils/AppError.js
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
    this.status     = statusCode >= 400 && statusCode < 500 ? 'fail' : 'error';
    this.isOperational = true;         // distinguish from programming errors
    Error.captureStackTrace(this, this.constructor);
  }
}

export default AppError;

// Usage inside any service:
// throw new AppError('Employee not found', 404);
// throw new AppError('Email already registered', 409);
```

## 5.2 catchAsync.js — Async Wrapper

Wrap every async controller function with catchAsync so you never need try/catch blocks inside controllers.

```
// src/utils/catchAsync.js
const catchAsync = (fn) => {
  return (req, res, next) => {
    fn(req, res, next).catch(next);   // any thrown error goes to globalErrorHandler
  };
};

export default catchAsync;

// Usage in controller:
// export const getEmployee = catchAsync(async (req, res) => { ... });
```

## 5.3 responseHandler.js — Standardised Responses

Every single API response must go through one of these two helpers. No ad-hoc res.json() calls are allowed outside of these utilities.

```
// src/utils/responseHandler.js
export const sendSuccess = (res, data, message = 'Success', statusCode = 200) => {
  res.status(statusCode).json({
    success : true,
    message,
    data,
  });
};

export const sendError = (res, message = 'Something went wrong', statusCode = 500) => {
  res.status(statusCode).json({
    success : false,
    message,
```

```
  });
};
```

### 5.4 errorHandler.js — Global Error Middleware

Registered as the last middleware in app.js. Catches all errors forwarded via next(err) or thrown inside catchAsync-wrapped functions.

```
// src/utils/errorHandler.js
import AppError from './AppError.js';

// Handle Supabase-specific errors
const handleSupabaseError = (err) => {
  if (err.code === '23505') return new AppError('Duplicate value — record already exists.',
409);
  if (err.code === '23503') return new AppError('Related record not found (foreign key).', 400);
  return new AppError('Database error.', 500);
};

// Handle JWT errors
const handleJWTError     = () => new AppError('Invalid token. Please log in again.', 401);
const handleJWTExpired   = () => new AppError('Token expired. Please log in again.', 401);

const sendErrorDev  = (err, res) => res.status(err.statusCode).json({
  success : false,
  status  : err.status,
  message : err.message,
  stack   : err.stack,
});

const sendErrorProd = (err, res) => {
  if (err.isOperational) {
    res.status(err.statusCode).json({ success: false, message: err.message });
  } else {
    console.error('UNHANDLED ERROR:', err);
    res.status(500).json({ success: false, message: 'Internal server error.' });
  }
};

export const globalErrorHandler = (err, req, res, next) => {
  err.statusCode = err.statusCode || 500;
  err.status     = err.status     || 'error';

  if (process.env.NODE_ENV === 'development') {
    sendErrorDev(err, res);
  } else {
    let error = { ...err, message: err.message };
    if (err.name === 'JsonWebTokenError')  error = handleJWTError();
    if (err.name === 'TokenExpiredError')  error = handleJWTExpired();
    if (err.code?.startsWith('23'))        error = handleSupabaseError(err);
    sendErrorProd(error, res);
  }
};
```

## 6. Configuration — Supabase & Upstash Redis

## 6.1 Supabase Client

```
// src/config/supabase.js
import { createClient } from '@supabase/supabase-js';

const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_SERVICE_KEY   // Service role key for server-side access
);

export default supabase;
```

## 6.2 Upstash Redis Client

Upstash provides a free-tier serverless Redis. Get your REST_URL and REST_TOKEN from https://console.upstash.com and add them to your .env file.

```
// src/config/redis.js
import { Redis } from '@upstash/redis';

const redis = new Redis({
  url:   process.env.UPSTASH_REDIS_REST_URL,
  token: process.env.UPSTASH_REDIS_REST_TOKEN,
});

export default redis;

// .env entries needed:
// UPSTASH_REDIS_REST_URL=https://xxxxx.upstash.io
// UPSTASH_REDIS_REST_TOKEN=AXxx...
```

## 6.3 Environment Variables (.env.example)

```
NODE_ENV=development
PORT=5000

# Supabase
SUPABASE_URL=https://xxxx.supabase.co
SUPABASE_SERVICE_KEY=eyJhbGci...

# JWT
JWT_SECRET=your_super_secure_random_string_min_32_chars
JWT_EXPIRES_IN=8h

# Upstash Redis
UPSTASH_REDIS_REST_URL=https://xxxx.upstash.io
UPSTASH_REDIS_REST_TOKEN=AXxx...

# CORS
ALLOWED_ORIGINS=http://localhost:3000,https://yourdomain.com
```

# 7. Middlewares (5 Marks)

## 7.1 CORS Middleware

```
// src/middlewares/cors.middleware.js
import cors from 'cors';

const allowedOrigins = process.env.ALLOWED_ORIGINS?.split(',') || [];

export const corsOptions = cors({
  origin: (origin, callback) => {
    // Allow requests with no origin (e.g. Postman, server-to-server)
    if (!origin || allowedOrigins.includes(origin)) {
      callback(null, true);
    } else {
      callback(new Error(`CORS: Origin '${origin}' not allowed`));
    }
  },
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization'],
});
```

## 7.2 Auth Middleware

```
// src/middlewares/auth.middleware.js
import jwt       from 'jsonwebtoken';
import redis     from '../config/redis.js';
import AppError  from '../utils/AppError.js';
import catchAsync from '../utils/catchAsync.js';

export const protect = catchAsync(async (req, res, next) => {
  // 1. Extract token from Authorization header
  const token = req.headers.authorization?.startsWith('Bearer ')
    ? req.headers.authorization.split(' ')[1]
    : null;

  if (!token) throw new AppError('You are not logged in. Please authenticate.', 401);

  // 2. Check Redis blacklist (logged-out tokens)
  const isBlacklisted = await redis.get(`bl_${token}`);
  if (isBlacklisted) throw new AppError('Token has been revoked. Please log in again.', 401);

  // 3. Verify token — will throw JsonWebTokenError or TokenExpiredError
  //    which are caught and handled in globalErrorHandler
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  req.user  = decoded;
  req.token = token;
  next();
});
```

## 7.3 Input Sanitiser Middleware

```
// src/middlewares/sanitise.middleware.js
import mongoSanitize from 'express-mongo-sanitize';
import xss           from 'xss-clean';
```

```
// Apply both globally in app.js: app.use(...sanitiseInputs)
export const sanitiseInputs = [
  mongoSanitize(),   // strips $ and . from keys to prevent NoSQL injection
  xss(),             // escapes HTML special characters in values
];
```

### 7.4 Validation Middleware Factory

```
// src/middlewares/validate.middleware.js
import AppError from '../utils/AppError.js';

// Usage: router.post('/register', validate(registerSchema), authController.register)
const validate = (schema) => (req, res, next) => {
  const { error } = schema.validate(req.body, { abortEarly: false });
  if (error) {
    const message = error.details.map((d) => d.message).join(', ');
    return next(new AppError(message, 400));
  }
  next();
};

export default validate;
```

## 8. Models

Models in this architecture are not ORM classes — they are plain JavaScript objects that document the shape of each entity. They are used for reference by other layers and can export validation schemas.

### 8.1 Employee Model

```
// src/models/employee.model.js
import Joi from 'joi';

// Documents the Supabase 'employees' table schema
export const employeeFields = {
  id:            'uuid (auto)',
  first_name:    'string, required',
  last_name:     'string, required',
  email:         'string, unique, required',
  phone:         'string, optional',
  department_id: 'uuid → departments.id',
  role_id:       'uuid → roles.id',
  hire_date:     'date, required',
  salary:        'numeric, required',
  is_active:     'boolean, default true',
  created_at:    'timestamp (auto)',
  updated_at:    'timestamp (auto)',
};

// Joi validation schema for create
export const createEmployeeSchema = Joi.object({
  first_name:    Joi.string().trim().min(2).max(60).required(),
  last_name:     Joi.string().trim().min(2).max(60).required(),
  email:         Joi.string().email().lowercase().required(),
```

```
  phone:         Joi.string().optional(),
  department_id: Joi.string().uuid().required(),
  role_id:       Joi.string().uuid().required(),
  hire_date:     Joi.date().iso().required(),
  salary:        Joi.number().positive().required(),
});
```

## 8.2 Other Models (same pattern)

- department.model.js — exports departmentFields + createDepartmentSchema
- role.model.js — exports roleFields + createRoleSchema
- attendance.model.js — exports attendanceFields + checkInSchema

# 9. Repositories — Data Access Layer

Repositories are the only layer that talks to Supabase. Services call repositories; repositories never call services. No req/res objects here.

## 9.1 employee.repository.js

```
// src/repositories/employee.repository.js
import supabase  from '../config/supabase.js';
import AppError  from '../utils/AppError.js';

const TABLE = 'employees';

export const findAll = async ({ page = 1, limit = 10 } = {}) => {
  const from = (page - 1) * limit;
  const to   = from + limit - 1;
  const { data, error, count } = await supabase
    .from(TABLE)
    .select('*, departments(name), roles(title)', { count: 'exact' })
    .eq('is_active', true)
    .range(from, to);
  if (error) throw new AppError(error.message, 500);
  return { data, count };
};

export const findById = async (id) => {
  const { data, error } = await supabase
    .from(TABLE)
    .select('*, departments(name), roles(title)')
    .eq('id', id)
    .single();
  if (error) throw new AppError('Employee not found', 404);
  return data;
};

export const findByDepartment = async (deptId) => {
  const { data, error } = await supabase
    .from(TABLE)
    .select('*, roles(title)')
    .eq('department_id', deptId)
    .eq('is_active', true);
  if (error) throw new AppError(error.message, 500);
  return data;
};

export const create = async (payload) => {
  const { data, error } = await supabase
    .from(TABLE).insert([payload]).select();
  if (error) throw new AppError(error.message, 500);
  return data[0];
};

export const update = async (id, payload) => {
  const { data, error } = await supabase
    .from(TABLE)
    .update({ ...payload, updated_at: new Date().toISOString() })
    .eq('id', id).select();
  if (error || !data.length) throw new AppError('Employee not found or update failed', 404);
  return data[0];
};
```

```
export const softDelete = async (id) => {
  const { data, error } = await supabase
    .from(TABLE)
    .update({ is_active: false, updated_at: new Date().toISOString() })
    .eq('id', id).select();
  if (error || !data.length) throw new AppError('Employee not found', 404);
  return data[0];
};
```

### 9.2 auth.repository.js

```
// src/repositories/auth.repository.js
import supabase from '../config/supabase.js';
import AppError from '../utils/AppError.js';

const TABLE = 'users';

export const findByEmail = async (email) => {
  const { data } = await supabase
    .from(TABLE).select('*').eq('email', email).single();
  return data;    // returns null if not found — no throw
};

export const createUser = async (payload) => {
  const { data, error } = await supabase
    .from(TABLE).insert([payload]).select();
  if (error) throw new AppError(error.message, 500);
  return data[0];
};

export const updatePassword = async (id, hashedPassword) => {
  const { error } = await supabase
    .from(TABLE)
    .update({ password: hashedPassword, updated_at: new Date().toISOString() })
    .eq('id', id);
  if (error) throw new AppError(error.message, 500);
};
```

You must also write: department.repository.js, role.repository.js, attendance.repository.js following the same pattern above.

## 10. Services — Business Logic Layer

Services orchestrate repositories and apply business rules. They have no knowledge of Express (req/res). All caching logic lives here.

### 10.1 auth.service.js

```
// src/services/auth.service.js
import bcrypt   from 'bcryptjs';
import jwt      from 'jsonwebtoken';
import redis    from '../config/redis.js';
```

```
import * as authRepo from '../repositories/auth.repository.js';
import AppError from '../utils/AppError.js';

export const register = async ({ name, email, password }) => {
  const exists = await authRepo.findByEmail(email);
  if (exists) throw new AppError('Email already registered.', 409);

  const hashed = await bcrypt.hash(password, 12);
  const user   = await authRepo.createUser({ name, email, password: hashed });

  const { password: _, ...safeUser } = user;  // strip password from response
  return safeUser;
};

export const login = async ({ email, password }) => {
  const user = await authRepo.findByEmail(email);
  if (!user) throw new AppError('Invalid email or password.', 401);

  const valid = await bcrypt.compare(password, user.password);
  if (!valid) throw new AppError('Invalid email or password.', 401);

  const token = jwt.sign(
    { id: user.id, email: user.email },
    process.env.JWT_SECRET,
    { expiresIn: process.env.JWT_EXPIRES_IN || '8h' }
  );
  return { token };
};

export const logout = async (token) => {
  const decoded = jwt.decode(token);
  const ttl = decoded.exp - Math.floor(Date.now() / 1000);
  if (ttl > 0) await redis.set(`bl_${token}`, '1', { ex: ttl });
};

export const changePassword = async (userId, { oldPassword, newPassword }) => {
  const user = await authRepo.findById(userId);
  const valid = await bcrypt.compare(oldPassword, user.password);
  if (!valid) throw new AppError('Current password is incorrect.', 401);
  const hashed = await bcrypt.hash(newPassword, 12);
  await authRepo.updatePassword(userId, hashed);
};
```

## 10.2 employee.service.js (with Redis caching)

```
// src/services/employee.service.js
import redis     from '../config/redis.js';
import * as repo from '../repositories/employee.repository.js';

const CACHE_TTL = 300;  // 5 minutes

// ── Helpers ──────────────────────────────────────────────────────
const invalidateListCache = async () => {
  // Upstash scan to clear all paginated list keys
  const { keys } = await redis.scan(0, { match: 'employees:list:*', count: 100 });
  if (keys.length) await redis.del(...keys);
};

// ── Service methods ──────────────────────────────────────────────
export const getAllEmployees = async (page, limit) => {
```

```
  const cacheKey = `employees:list:${page}:${limit}`;
  const cached   = await redis.get(cacheKey);
  if (cached) return cached;                          // Upstash returns parsed JSON

  const result = await repo.findAll({ page, limit });
  await redis.set(cacheKey, result, { ex: CACHE_TTL });
  return result;
};

export const getEmployeeById = async (id) => {
  const cacheKey = `employees:single:${id}`;
  const cached   = await redis.get(cacheKey);
  if (cached) return cached;

  const data = await repo.findById(id);
  await redis.set(cacheKey, data, { ex: CACHE_TTL });
  return data;
};

export const createEmployee = async (payload) => {
  const data = await repo.create(payload);
  await invalidateListCache();
  return data;
};

export const updateEmployee = async (id, payload) => {
  const data = await repo.update(id, payload);
  await redis.del(`employees:single:${id}`);
  await invalidateListCache();
  return data;
};

export const deleteEmployee = async (id) => {
  const data = await repo.softDelete(id);
  await redis.del(`employees:single:${id}`);
  await invalidateListCache();
  return data;
};
```

# 11. Controllers — Request / Response Layer

Controllers are thin. They extract data from req, call the service, and return a response using sendSuccess. Error forwarding is handled automatically by catchAsync.

## 11.1 auth.controller.js

```
// src/controllers/auth.controller.js
import catchAsync          from '../utils/catchAsync.js';
import { sendSuccess }     from '../utils/responseHandler.js';
import * as authService    from '../services/auth.service.js';

export const register = catchAsync(async (req, res) => {
  const user = await authService.register(req.body);
  sendSuccess(res, user, 'Account created successfully', 201);
});

export const login = catchAsync(async (req, res) => {
  const result = await authService.login(req.body);
  sendSuccess(res, result, 'Login successful');
});

export const getMe = catchAsync(async (req, res) => {
  sendSuccess(res, req.user, 'User profile');
});

export const logout = catchAsync(async (req, res) => {
  await authService.logout(req.token);
  sendSuccess(res, null, 'Logged out successfully');
});

export const changePassword = catchAsync(async (req, res) => {
  await authService.changePassword(req.user.id, req.body);
  sendSuccess(res, null, 'Password updated successfully');
});
```

## 11.2 employee.controller.js

```
// src/controllers/employee.controller.js
import catchAsync             from '../utils/catchAsync.js';
import { sendSuccess }        from '../utils/responseHandler.js';
import * as employeeService   from '../services/employee.service.js';

export const getAll = catchAsync(async (req, res) => {
  const page   = Number(req.query.page)  || 1;
  const limit  = Number(req.query.limit) || 10;
  const result = await employeeService.getAllEmployees(page, limit);
  sendSuccess(res, result, 'Employees retrieved');
});

export const getById = catchAsync(async (req, res) => {
  const data = await employeeService.getEmployeeById(req.params.id);
  sendSuccess(res, data, 'Employee retrieved');
});

export const create = catchAsync(async (req, res) => {
  const data = await employeeService.createEmployee(req.body);
```

```
  sendSuccess(res, data, 'Employee created', 201);
});

export const update = catchAsync(async (req, res) => {
  const data = await employeeService.updateEmployee(req.params.id, req.body);
  sendSuccess(res, data, 'Employee updated');
});

export const remove = catchAsync(async (req, res) => {
  await employeeService.deleteEmployee(req.params.id);
  sendSuccess(res, null, 'Employee deactivated');
});
```

# 12. Routes

## 12.1 auth.routes.js

```
// src/routes/auth.routes.js
import { Router }          from 'express';
import * as authController from '../controllers/auth.controller.js';
import { protect }         from '../middlewares/auth.middleware.js';
import validate            from '../middlewares/validate.middleware.js';
import { registerSchema, loginSchema } from '../models/employee.model.js';

const router = Router();

router.post('/register',         validate(registerSchema),  authController.register);
router.post('/login',            validate(loginSchema),     authController.login);
router.get( '/me',               protect,                   authController.getMe);
router.post('/logout',           protect,                   authController.logout);
router.put( '/change-password',  protect,                   authController.changePassword);

export default router;
```

## 12.2 employee.routes.js

```
// src/routes/employee.routes.js
import { Router }               from 'express';
import * as employeeController  from '../controllers/employee.controller.js';
import { protect }              from '../middlewares/auth.middleware.js';
import validate                 from '../middlewares/validate.middleware.js';
import { createEmployeeSchema } from '../models/employee.model.js';

const router = Router();

router.use(protect);   // All employee routes require authentication

router.get( '/',    employeeController.getAll);
router.get( '/:id', employeeController.getById);
router.post('/',    validate(createEmployeeSchema), employeeController.create);
router.put( '/:id', employeeController.update);
router.delete('/:id', employeeController.remove);

export default router;
```

## 12.3 Root Router — routes/index.js

```
// src/routes/index.js
import { Router }      from 'express';
import authRoutes       from './auth.routes.js';
import employeeRoutes   from './employee.routes.js';
import departmentRoutes from './department.routes.js';
import roleRoutes       from './role.routes.js';
import attendanceRoutes from './attendance.routes.js';

const router = Router();

router.use('/auth',        authRoutes);
router.use('/employees',   employeeRoutes);
router.use('/departments', departmentRoutes);
router.use('/roles',       roleRoutes);
router.use('/attendance',  attendanceRoutes);

export default router;
```

# 13. Required API Endpoints

## 13.1 Auth — /api/auth

| Method | Endpoint | Description | Auth |
|---|---|---|---|
| POST | /api/auth/register | Register a new user (hashed password) | Public |
| POST | /api/auth/login | Validate credentials, return JWT | Public |
| GET | /api/auth/me | Return authenticated user's profile | **Required** |
| POST | /api/auth/logout | Blacklist token in Upstash Redis | **Required** |
| PUT | /api/auth/change-password | Verify old password, save new hash | **Required** |

## 13.2 Employees — /api/employees

| Method | Endpoint | Description | Auth |
|---|---|---|---|
| GET | /api/employees | List all active employees (paginated, cached) | **Required** |
| GET | /api/employees/:id | Get single employee (cached) | **Required** |
| GET | /api/employees/department/:deptId | Filter by department | **Required** |
| POST | /api/employees | Create new employee | **Required** |
| PUT | /api/employees/:id | Update employee details | **Required** |
| DELETE | /api/employees/:id | Soft delete (set is_active = false) | **Required** |

## 13.3 Departments — /api/departments

| Method | Endpoint | Description | Auth |
|---|---|---|---|
| GET | /api/departments | List all departments (cached) | **Required** |
| GET | /api/departments/:id | Get department + employee count (cached) | **Required** |
| POST | /api/departments | Create new department | **Required** |
| PUT | /api/departments/:id | Update department name or manager | **Required** |
| DELETE | /api/departments/:id | Delete only if no active employees linked | **Required** |

## 13.4 Roles — /api/roles

| Method | Endpoint | Description | Auth |
|---|---|---|---|
| GET | /api/roles | List all roles (cached) | **Required** |
| GET | /api/roles/:id | Get single role with linked employees | **Required** |

| | | | |
|---|---|---|---|
| POST | /api/roles | Create new role | Required |
| PUT | /api/roles/:id | Update role details | Required |
| DELETE | /api/roles/:id | Delete only if no employees assigned | Required |

### 13.5 Attendance — /api/attendance & /api/leave

| Method | Endpoint | Description | Auth |
|---|---|---|---|
| GET | /api/attendance | List records with date range filter | Required |
| GET | /api/attendance/employee/:id | Records for specific employee | Required |
| POST | /api/attendance/check-in | Mark check-in for today | Required |
| POST | /api/attendance/check-out | Mark check-out, compute hours_worked | Required |
| POST | /api/leave | Submit a leave request | Required |
| PUT | /api/leave/:id | Approve or reject a leave request | Required |
| GET | /api/leave/employee/:id | Leave history for an employee | Required |

## 14. Supabase Database Schema

Create all tables in Supabase before writing application code. Use the Supabase table editor or SQL editor.

```
-- Run in Supabase SQL Editor

create extension if not exists "uuid-ossp";

create table departments (
  id          uuid primary key default uuid_generate_v4(),
  name        text unique not null,
  description text,
  manager_id  uuid,
  created_at  timestamptz default now(),
  updated_at  timestamptz default now()
);

create table roles (
  id            uuid primary key default uuid_generate_v4(),
  title         text not null,
  department_id uuid references departments(id),
  min_salary    numeric,
  max_salary    numeric,
  level         text check (level in ('Junior','Mid','Senior','Lead')),
  created_at    timestamptz default now(),
  updated_at    timestamptz default now()
);

create table employees (
  id            uuid primary key default uuid_generate_v4(),
  first_name    text not null,
  last_name     text not null,
```

```
  email         text unique not null,
  phone         text,
  department_id uuid references departments(id),
  role_id       uuid references roles(id),
  hire_date     date not null,
  salary        numeric not null,
  is_active     boolean default true,
  created_at    timestamptz default now(),
  updated_at    timestamptz default now()
);

create table users (
  id         uuid primary key default uuid_generate_v4(),
  name       text not null,
  email      text unique not null,
  password   text not null,
  created_at timestamptz default now(),
  updated_at timestamptz default now()
);

create table attendance (
  id          uuid primary key default uuid_generate_v4(),
  employee_id uuid references employees(id),
  date        date not null,
  check_in    timestamptz,
  check_out   timestamptz,
  hours_worked numeric,
  status      text check (status in ('present','absent','half-day','leave'))
);

-- Add FK for department manager after employees table exists
alter table departments
  add constraint fk_manager foreign key (manager_id) references employees(id);
```

# 15. Marking Rubric

| Area | What is Evaluated | Marks |
|---|---|---|
| **Project Structure & Clean Architecture** | Correct folder layout, separation of concerns, no logic in routes | **15** |
| **Authentication Module** | Register, login, JWT, bcrypt, auth middleware, blacklist | **20** |
| **CRUD — Employees** | All 5+ endpoints, model, repo, service, controller wired | **15** |
| **CRUD — Departments** | All endpoints, business rules enforced | **10** |
| **CRUD — Roles & Positions** | All endpoints, validation, service logic | **10** |
| **CRUD — Attendance & Leave** | Check-in/out, leave flow, status enum | **10** |
| **Middleware (Auth, CORS, Sanitiser)** | Custom implementation, correct placement | **05** |
| **Supabase ORM** | Schema design, relations, joins, pagination | **05** |
| **Upstash Redis Caching** | Cache-aside, TTL, invalidation on mutation | **05** |
| **Error Handling & Response Handler** | Global error handler, AppError class, consistent responses | **05** |
| **TOTAL** | | **100** |

# 16. Submission Instructions

## What to Submit

- A GitHub repository (public or shared) with the full source code.
- A Postman Collection (.json) with all endpoints, sample request bodies and expected responses.
- A README.md with: project description, folder structure explanation, environment variable list (.env.example), and setup steps.
- A short written summary (max 1 page) describing your Redis caching strategy and any decisions you made.

## Submission Checklist

1. node_modules/ is excluded via .gitignore.
2. .env is gitignored; .env.example is committed.
3. package.json includes "type": "module".
4. All four CRUD modules have working endpoints (verified via Postman).
5. Auth register → login → protected route → logout flow works end to end.
6. Redis caching is demonstrable: log [CACHE HIT] vs [DB QUERY] in console.
7. globalErrorHandler returns consistent JSON for all error types.
8. Middleware is applied correctly (CORS globally, auth per route, sanitiser globally).

9. Supabase schema has all foreign key relationships defined.
10. README is complete and a reviewer can set up the project from scratch.

## Tips for Success

- Set up Supabase and create all tables first — run the SQL from Section 14.
- Build and test the utility files (AppError, catchAsync, responseHandler) before any feature.
- Use Postman to test each endpoint as you build it, not all at the end.
- Add console.log('[CACHE HIT]') and console.log('[DB QUERY]') while developing to verify caching.
- Commit to GitHub regularly with meaningful messages — examiners review commit history.
- Read the Upstash docs — the @upstash/redis client auto-serialises JSON, no JSON.stringify needed.

**Good luck! Build something you are proud of.**

*— Asyntax Coding School, Backend Development Faculty*